



Streams & Files in C++



The <fstream> Header & Stream Classes

#include <fstream> brings in the three file-stream classes



ifstream *input file stream*

Reads FROM a file. Inherits istream, so it has >>, get(), getline(), read().



ofstream *output file stream*

Writes TO a file. Inherits ostream, so it has <<, put(), write().



fstream *input + output*

Reads AND writes one file. Inherits istream (both directions).



```
#include <fstream>
#include <iostream>
using namespace std;

ifstream fin("data.txt");    // read
ofstream fout("out.txt");    // write
fstream fio("rec.dat");      // both
```

Class hierarchy

ifstream derives from **istream**,
ofstream from **ostream**, and
fstream from **iostream**. So the operators you know from
cin/cout work on files too.



Opening Files & File Modes

The second argument to a constructor / `open()` is a combination of `ios::` flags



`ios::in`

Open for reading



`ios::out`

Open for writing



`ios::app`

Append — every write goes to the END;
existing content is KEPT



`ios::trunc`

Truncate — existing content is ERASED on
open



`ios::ate`

Open and seek to end (you can still write
anywhere)



`ios::binary`

Binary mode — no newline / text
translation

app vs trunc — *the key contrast*

`ios::app` preserves the file and forces writes to the end; `ios::trunc` wipes the file clean the moment it opens.



```
// append: keeps old text, adds at end
ofstream f1("log.txt", ios::app);

// truncate: erase, then write fresh
ofstream f2("data.txt", ios::trunc);

// combine flags with bitwise OR
fstream f3("rec.dat",
           ios::in | ios::out | ios::binary);
```



File Position Pointers

Every open file tracks where the next read / write will happen

get pointer (input)

Marks where the NEXT READ starts. Moved by `seekg()`, reported by `tellg()`.

put pointer (output)

Marks where the NEXT WRITE goes. Moved by `seekp()`, reported by `tellp()`.

Seek directions: `ios::beg` (from start) `ios::cur` (from current) `ios::end` (from end)

```
fin.seekg(10);           // get ptr moves byte 10 from beginning
fin.seekg(-5, ios::end);  // get ptr moves 5 bytes before end
streampos g = fin.tellg(); // current read position

fout.seekp(0, ios::beg);  // put ptr moves to start of file
streampos p = fout.tellp(); // current write position
```

Random-Access File I/O in C++

Scenario: a library stores book records in `library.dat`. Each record (an object of a class) is exactly 40 bytes. We have an array of records. Index of first record is zero.

library.dat — the code listing

```
1  fstream file("library.dat",  
    ios::in | ios::out | ios::binary);  
  
2  file.seekp(160, ios::beg);  
3  file.write(newRecord, 40);  
4  cout << file.tellp() << endl;  
  
5  file.seekg(80, ios::beg);  
6  file.read(buffer, 40);  
7  cout << file.tellg() << endl;
```

Short Answers



i. Which record no is written in file

$160 \div 40$

Record 4



ii. Which record no is read from file

$80 \div 40$

Record 2

iii. Printed output

Line 4 **200**

Line 7 **120**

Key Concepts Explained

How each line moves the file pointers — and why the answers come out the way they do.



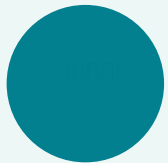
Binary read/write mode

`ios::in` | `ios::out` | `ios::binary` opens one file for BOTH reading and writing, with no text translation — bytes are stored exactly. `ios::beg` means offsets are measured from the start of the file.



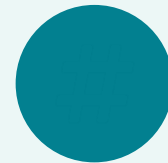
Two separate pointers

An `fstream` keeps a put pointer (where the next write lands) and a get pointer (where the next read starts). `seekp/tellp` control the put pointer; `seekg/tellg` control the get pointer. They move independently.



Record number formula

Records are fixed-length, so $\text{byte offset} = \text{record number} \times \text{record size}$. To find the record from an offset, reverse it: $\text{record} = \text{offset} \div \text{size}$. Here $160 \div 40 = \text{record 4}$ (write); $80 \div 40 = \text{record 2}$ (read).



read / write advance the pointer

After `write(newRecord, 40)` the put pointer moves 40 bytes on: $160 + 40 = 200$, so `tellp` prints 200. After `read(buffer, 40)` the get pointer moves 40 bytes on: $80 + 40 = 120$, so `tellg` prints 120.

Remember: $\text{offset} \rightarrow \text{record} = \text{offset} \div \text{size}$ · `tellp` / `tellg` report the pointer AFTER the operation finishes.



Your Turn — What Gets Printed?

Trace the position pointers and give the value at each (A)–(D)



```
ofstream fout("test.txt");
fout << "HelloWorld";           // 10 chars
cout << fout.tellp();           // (A) ?
fout.seekp(5);
cout << fout.tellp();           // (B) ?
fout.close();

ifstream fin("test.txt");
fin.seekg(0, ios::end);
cout << fin.tellg();            // (C) ?
fin.seekg(3, ios::beg);
cout << fin.tellg();            // (D) ?
```

A

after writing 10 chars

= _____

B

after seekp(5)

= _____

C

after seekg(0, end)

= _____

D

after seekg(3, beg)

= _____



Writing a Class Object to a File (Binary)

Dump the object's raw bytes with `ostream::write()`

The recipe

- 1 Open in binary** Add `ios::binary` so bytes are stored verbatim (no newline translation).
- 2 Call `write()`** `write(const char* buf, streamsize n)` copies `n` raw bytes from `buf` to the file.
- 3 Reinterpret the address** `write()` wants a `char*`, so pass `reinterpret_cast<char*>(&obj)`.
- 4 Size it correctly** Use `sizeof(obj)` so exactly one whole object is written.

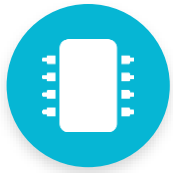


```
class Student {
public:
    int    roll;
    char   name[20];    // fixed size
    float  gpa;
};

Student s{ 101, "Ayesha", 3.8f };
ofstream fout("students.dat", ios::binary);
fout.write(reinterpret_cast<char*>(&s),
           sizeof(s));
```



Use trivially-copyable types only. Prefer `char name[20]` over `std::string` — a string stores an internal pointer, so writing it dumps the pointer, not the text.



reinterpret_cast — Why Binary I/O Needs It

Tell the compiler to view an object's address as a stream of raw bytes

The type mismatch it solves

`write()` and `read()` are declared to take a `char*` buffer. But `&s` has type `Student*` — an unrelated pointer type. `reinterpret_cast` bridges them.

→ **Compile-time only** Pure type reinterpretation — no code runs, no conversion of the value.

→ **Address is unchanged** Same memory address; only the pointer's type label changes.

→ **No checks, no overhead** It trusts you completely — fast, but unsafe if misused.

```
// &s is Student* ; write() wants char*
fout.write(
    reinterpret_cast<char*>(&s),
    sizeof(s));

// reading the bytes back into an object
Student s2;
fin.read(
    reinterpret_cast<char*>(&s2),
    sizeof(s2));
```

Mental model: “Don't convert anything — just look at the same memory through `char*` glasses so I can copy it byte by byte.”



reinterpret_cast vs dynamic_cast

Same keyword family, completely different jobs

Aspect	reinterpret_cast	dynamic_cast
What it does	Reinterprets a pointer's bit pattern as another type	Safely converts within a polymorphic class hierarchy
Checked	Compile-time only — NO runtime check	At runtime, using RTTI
Requirement	None	Base class needs at least one virtual function
On failure	Undefined behaviour if misused	nullptr (pointers) / throws bad_cast (references)
Typical use	Binary file I/O: write() / read() buffers	Downcasting base -> derived safely

In one line: `reinterpret_cast` is about **bytes** (no questions asked); `dynamic_cast` is about **object identity** in an inheritance tree (verified at runtime).



Worked Example — Writing Many Objects

Each record is sizeof(Student) bytes, written back-to-back

```
Student list[3] = {
    {101, "Ayesha", 3.8f},
    {102, "Bilal", 3.5f},
    {103, "Sara", 3.9f}
};

fstream f("students.dat",
        ios::out | ios::binary);

for (int i = 0; i < 3; ++i)
    f.write(reinterpret_cast<char*>(&list[i]),
           sizeof(Student));
f.close();
```

Resulting file layout

0

Rec 0 roll 101 · Ayesha

bytes 0 - 27

1

Rec 1 roll 102 · Bilal

bytes 28 - 55

2

Rec 2 roll 103 · Sara

bytes 56 - 83

Assuming sizeof(Student) = 28 bytes



Worked Example — Reading One Specific Object

Random access: jump straight to record k , then read it

```
int k = 2;           // want the 3rd student
Student s;

fstream f("students.dat", ios::in | ios::binary);

// offset = record index x record size
f.seekg(k * sizeof(Student), ios::beg);
f.read(reinterpret_cast<char*>(&s),
        sizeof(s));

cout << s.roll << " " << s.name << " " << s.gpa;
f.close();
```

The key formula

offset = $k \times \text{sizeof}(\text{Student})$

For $k = 2$: offset = $2 \times 28 = 56$ bytes

Why this works

Fixed-size records mean any object is one **seekg** + one **read** away — no need to scan from the start. This is the basis of indexed record files.



How Many Objects Are in the File?

File size ÷ size of one object = number of records

```
fstream f("students.dat", ios::in | ios::binary);

f.seekg(0, ios::end);          // jump to end
streampos bytes = f.tellg(); // = file size

int count = bytes / sizeof(Student);
cout << "Records: " << count;
f.close();
```

Worked numbers

280 bytes total

÷

28 bytes / object

= 10 records



Caveat: this only works when every record is exactly the same size. If the class stored variable-length data (e.g. a real `std::string`), records would differ in size and the division would be meaningless.



EOF, Stream State & Key Functions

How a stream reports trouble — and the members you'll reach for

State flags



eofbit

End of file reached



failbit

Logical error (e.g. read wrong type)



badbit

Serious / unrecoverable I/O error



goodbit

All clear, no error

Checking & clearing state

`eof()` · `fail()` · `bad()` · `good()` query the flags;
`clear()` resets them so the stream is usable again. The stream also converts to bool, so `if (fin)` tests `good()`.

Other useful members

`open()` `close()` `is_open()` `get()` `getline()` `put()`
`read()` `write()` `peek()` `ignore()` `flush()`



```
// PITFALL: tests EOF one read too late
while (!fin.eof()) { // avoid!
    fin >> x; cout << x;
}

// CORRECT: loop on the read itself
while (fin >> x)
    cout << x;    // stops at EOF/fail
```



Overloading << and >> Operators

Teach the stream how to read and print YOUR types

`operator<<` (insertion / output) and `operator>>` (extraction / input) let you write `cout << p;` and `cin >> p;` for a custom class.

```
class Point {
    int x, y;
public:
    Point(int a=0, int b=0):x(a),y(b){}
    friend ostream& operator<<(ostream&, const Point&);
    friend istream& operator>>(istream&, Point&);
};

ostream& operator<<(ostream& out, const Point& p){
    out << p.x << "," << p.y;
    return out;           // enables chaining
}

istream& operator>>(istream& in, Point& p){
    in >> p.x >> p.y; return in;
}
```

Return the stream by reference

Returning `ostream&` is what makes `cout << a << b << c;` chain left-to-right.

Signature pattern

Output: `const T&` (read-only).

Input: `T&` (modified, non-const).



Why They Can't Be Class Members

The left operand decides everything



If it were a member function...

A member operator's left operand is always ***this** — the class object. So you'd be forced to write **p << cout**; which is backwards and unnatural.



As a non-member (friend)...

The **stream** is the left operand, giving the natural **cout << p**; friend grants access to the private members it needs.



The decisive reason: the left operand **cout** / **cin** belongs to **ostream** / **istream** — standard-library classes you cannot modify. You can't add a member to them, so the operator must live outside as a free function.



```
cout << p;    // is really:  operator<<(cout, p);    <- stream on the left
p << cout;    // would be:  p.operator<<(cout);      <- member form, unnatural
```



Key Takeaways

1

Three classes, one header

ifstream reads, ofstream writes, fstream does both — from `<fstream>`.

2

Modes & pointers

`ios::app` keeps & appends, `ios::trunc` erases; `seekg/tellg` & `seekp/tellp` track byte positions.

3

Binary objects

`write()/read()` copy raw bytes via `reinterpret_cast<char*>(&obj)` — use trivially-copyable types.

4

Right cast, right job

`reinterpret_cast` = bit relabeling for I/O; `dynamic_cast` = runtime-checked downcast.

5

Random access & count

Record `k` is at `k*sizeof`; record count = `fileSize / sizeof(obj)`.

6

Operators as friends

`<<` / `>>` are non-member so the stream is the left operand; friend reaches privates.